

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

«Программирование»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

Итераторы, генераторы и декораторы

Выполнил:

Студент группы N3150

Пахомов Владимир Юрьевич



Проверил:

Грозов В. А.

Санкт-Петербург

2025г.

Оглавление

Задание. Вариант 2-2-8-1.....	3
Краткое описание работы алгоритма «регистр сдвига (xor-shift)».....	4
Пример работы программы при различных исходных данных.....	5
Исходный текст программы с комментариями.....	6

Задание. Вариант 2-2-8-1

Таблица 1. Алгоритм генерации псевдослучайных чисел:

2 — регистр сдвига (xor_shift)

Таблица 2. Способ реализации ГСПЧ:

2 — Генератор (yield)

Таблица 3. Действие, выполняемое декоратором:

8 — @shuffle_str_ret Декоратор должен заменить любые строки, которые передаются как позиционные или именованные аргументы в функцию или метод, на строки, в которых символы исходных строк переставлены в случайном (с помощью ГПСЧ) порядке.

Таблица 4. Способ реализации декоратора:

1 — Функция

Краткое описание работы алгоритма «регистр сдвига (xor-shift)»

Шаг 1: $x_{i+1} \wedge= x_i \gg a$,

Шаг 2: $x_{i+1} \wedge= x_{i+1} \ll b$,

Шаг 3: $x_{i+1} \wedge= x_{i+1} \gg c$,

где $\wedge=$, $\ll$ и $\gg$ — операции исключающего ИЛИ с присваиванием, сдвига влево и

сдвига вправо. В программе реализован 64-битный сдвиг с коэффициентами $a = 13$, $b = 7$, $c = 17$ по умолчанию.

Пример работы программы при различных исходных данных

```
>>>
(venv) vladimirp@vladimirp: ~/Py_Workspace/LR6/prg6vypN3150 python
Python 3.13.3 (main, Apr  9 2025, 07:44:25) [GCC 14.2.1 20250207] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from prg6vypN3150_rnd import xor_shift, MyPRNG
>>> from prg6vypN3150_subst import shuffle_str_ret
>>> prng = MyPRNG(xor_shift(12345))
>>> @shuffle_str_ret(prng, "test.log")
... def foo(name, surname):
...     return name + " " + surname
...
>>>
>>> print(foo("Vova", "Pakhomov"))
oVav ookvmPha
>>> prng.next_int()
2303841250356525524
>>> print(foo("Vova", "Pakhomov"))
aovV kmPaovoh
>>> prng.next_float()
0.5861764617203356
>>> prng.next_str(40)
'ar7cdEh+ugZWxE"b>u$ ]+fy|[3zPUm]PR|q0rhz'
>>> prng.shuffle_str("ABCDEFGF")
'CBFGFAD'
```

```
>>> @shuffle_str_ret(prng)
... def boo(**kwargs):
...     return str(kwargs)
...
>>> print(boo(name="Vova", surname="Pakhomov", age=18, height=187))
24.05.25 19:04:25 boo(args=(), kwargs={'name': 'Vova', 'surname': 'Pakhomov', 'age': 18, 'height': 187}) -> {'name': 'oVva', 'surname': 'ovhPmako', 'age': 18, 'height': 187}
24.05.25 19:04:25 boo: аргумент Vova (kwargs["name"]) заменён на {'name': 'oVva'}
24.05.25 19:04:25 boo: аргумент Pakhomov (kwargs["surname"]) заменён на {'surname': 'ovhPmako'}
{'name': 'oVva', 'surname': 'ovhPmako', 'age': 18, 'height': 187}
```

Исходный текст программы с комментариями

Файл prg6vuyrN3150_rnd.py:

```
# Вариант 2-2-8-1

# 2 - Алгоритм генерации псевдослучайных чисел
#   регистр сдвига (xor-shift)
#   Шаг 1:  $x_{i+1} \wedge= x_i \gg a$ 
#   Шаг 2:  $x_{i+1} \wedge= x_i \ll b$ 
#   Шаг 3:  $x_{i+1} \wedge= x_i \gg c$ 
#    $\wedge=$  - исключающее или с присваиванием
#    $\ll$  - сдвиг влево
#    $\gg$  - сдвиг вправо

# 2 - Способ реализации ГПСЧ
#   Генератор (yield)

# 8 - Действие, выполняемое декоратором
#   @shuffle_str_ret Декоратор должен заменить любые строки, которые
#   передаются как позиционные или именованные
#   аргументы в функцию или метод, на строки, в
#   которых символы исходных строк переставлены в
#   случайном (с помощью ГПСЧ) порядке.

# 1 - Способ реализации декоратора
#   Функция

from abc import ABC, abstractmethod
from collections.abc import Generator

# В этом модуле должно содержаться определение ГПСЧ -
# либо в виде генератора:
def xor_shift(state: int = 1, a: int = 13, b: int = 7, c: int = 17) -> Generator[int]:
    # Замечание: аннотацию Iterator[int] с Python 3.13 можно заменить на Generator[int]
    while True:
        state ^= (state << a) & 0xFFFFFFFFFFFFFFFF
        state ^= (state >> b) & 0xFFFFFFFFFFFFFFFF
        state ^= (state << c) & 0xFFFFFFFFFFFFFFFF
        yield state & 0xFFFFFFFFFFFFFFFF
```

```

class PRNGBase(ABC):
    @abstractmethod
    def next_int(self) -> int:
        """Возвращает случайное целое число"""
        ...

    @abstractmethod
    def next_float(self) -> float:
        """Возвращает случайное вещественное число в диапазоне от 0.0 до 1.0 (не включая)"""
        ...

    @abstractmethod
    def next_str(self) -> str:
        """Возвращает случайную последовательность символов"""
        ...

    @abstractmethod
    def shuffle_str(self, src: str) -> str:
        """Получает строку src и возвращает строку, содержащую символы из src, переставленные в случайном порядке"""
        ...

```

```

class MyPRNG(PRNGBase):
    def __init__(self, gen: Generator):
        self.gen = iter(gen)

    def next_int(self) -> int:
        return next(self.gen)

    def next_float(self) -> float:
        res: float = next(self.gen) / 2**64
        return res

    def next_str(self, length: int = 10) -> str:
        def random_chr():
            return chr(int(self.next_float() * (127 - 32) + 32))
        res: str = ''.join(random_chr() for _ in range(length))
        return res

    def shuffle_str(self, src: str) -> str:
        res: str = ""
        while len(src) > 0:

```

```
c: int = int(self.next_int() % len(src))
res += src[c]
src = src[:c] + src[c+1:]
return res
```


Файл prg6vypN3150_subst.py:

```
from prg6vypN3150_rnd import MyPRNG, xor_shift      # Импортировать ваш ГПСЧ и класс MyPRNG
from prg6vypN3150_subst import shuffle_str_ret      # Импортировать ваш декоратор
import pytest

# Тут ваши тесты

@pytest.fixture
def prng() -> MyPRNG:
    return MyPRNG(xor_shift(124))

def test_decorator_args(prng: MyPRNG):
    @shuffle_str_ret(prng)
    def greeting(name, surname):
        return f"Hello, {name} {surname}!"

    name, surname = "Vladimir", "Pakhomov"
    res = greeting(name, surname)
    assert sorted(res) == sorted(f"Hello, {name} {surname}!")
    assert res != f"Hello, {name} {surname}!"

def test_decorator_kwargs(prng: MyPRNG):
    @shuffle_str_ret(prng)
    def combine_data(hostname="", login="", password=""):
        return {key: value for key, value in {
            "hostname": hostname,
            "login": login,
            "password": password
        }.items()}

    data = {
        "hostname": "localhost",
        "login": "vladimir",
        "password": "12345",
    }
    res = combine_data(data["hostname"], data["login"], data["password"])
    assert res != data

def test_decorator_ignore_non_str(prng: MyPRNG):
    @shuffle_str_ret(prng)
    def sum_int(*args):
        return f"sum = {sum(x for x in args if isinstance(x, int))} from {args}"
```

```
data = (1, 2, 3, 4, 5, "123")
res = sum_int(*data)
assert res != f"sum = {sum(x for x in data if isinstance(x, int))} from {data}"
```

```
def test_decorator_with_class(prng: MyPRNG):
```

```
    @shuffle_str_ret(prng)
```

```
    class Funcs:
```

```
        def __init__(self, *args):
```

```
            self.args = list(*args)
```

```
        def get_args(self):
```

```
            return self.args
```

```
        def update_args(self, *new_args):
```

```
            self.args.extend(new_args)
```

```
            return self.args
```

```
data = [1, 2, 3, "231", "True"]
```

```
f = Funcs(data)
```

```
assert f.get_args() == data
```

```
new_data = ("Hello", "World")
```

```
res = f.update_args(*new_data)
```

```
data.extend(new_data)
```

```
assert res != data
```

```
# Количество тестов выбирайте самостоятельно,  
# но так, чтобы вся необходимая функциональность  
# была проверена.
```

Файл test_prg6vypN3150_rnd.py:

```
from prg6vypN3150_rnd import MyPRNG, xor_shift  # Импортировать ваш класс и ГПСЧ (вместо lcg
# должен быть алгоритм, зависящий от варианта)
import pytest
```

```
# Тут ваши тесты
```

```
@pytest.fixture
```

```
def prng() -> MyPRNG:
```

```
    return MyPRNG(xor_shift(12345))
```

```
def test_next_int(prng: MyPRNG):
```

```
    value = prng.next_int()
```

```
    assert isinstance(value, int)
```

```
def test_next_float(prng: MyPRNG):
```

```
    value = prng.next_float()
```

```
    assert 0 <= value < 1
```

```
def test_next_str(prng: MyPRNG):
```

```
    l = 100
```

```
    value = prng.next_str(l)
```

```
    assert isinstance(value, str)
```

```
    assert len(value) == l
```

```
def test_shuffle_str(prng: MyPRNG):
```

```
    src_str = "Hello, World!"
```

```
    value = prng.shuffle_str(src_str)
```

```
    assert isinstance(value, str)
```

```
    assert sorted(src_str) == sorted(src_str)
```

```
    assert len(value) == len(src_str)
```

```
    assert value != src_str
```

```
# Количество тестов выбирайте самостоятельно,
# но так, чтобы вся необходимая функциональность
# была проверена.
```

Файл test_prg6vypN3150_subst.py:

```
from prg6vypN3150_rnd import MyPRNG, xor_shift # Импортировать ваш ГПСЧ и класс MyPRNG
from prg6vypN3150_subst import shuffle_str_ret # Импортировать ваш декоратор
import pytest
```

```
# Тут ваши тесты
```

```
@pytest.fixture
```

```
def prng() -> MyPRNG:
```

```
    return MyPRNG(xor_shift(124))
```

```
def test_decorator_args(prng: MyPRNG):
```

```
    @shuffle_str_ret(prng)
```

```
    def greeting(name, surname):
```

```
        return f"Hello, {name} {surname}!"
```

```
    name, surname = "Vladimir", "Pakhomov"
```

```
    res = greeting(name, surname)
```

```
    assert sorted(res) == sorted(f"Hello, {name} {surname}!")
```

```
    assert res != f"Hello, {name} {surname}!"
```

```
def test_decorator_kwargs(prng: MyPRNG):
```

```
    @shuffle_str_ret(prng)
```

```
    def combine_data(hostname="", login="", password=""):
```

```
        return {key: value for key, value in {
```

```
            "hostname": hostname,
```

```
            "login": login,
```

```
            "password": password
```

```
        }.items()
```

```
    }
```

```
    data = {
```

```
        "hostname": "localhost",
```

```
        "login": "vladimir",
```

```
        "password": "12345",
```

```
    }
```

```
    res = combine_data(data["hostname"], data["login"], data["password"])
```

```
    assert res != data
```

```
def test_decorator_ignore_non_str(prng: MyPRNG):
```

```
    @shuffle_str_ret(prng)
```

```
    def sum_int(*args):
```

```
        return f"sum = {sum(x for x in args if isinstance(x, int))} from {args}"
```

```
    data = (1, 2, 3, 4, 5, "123")
```

```
    res = sum_int(*data)
```

```
assert res != f"sum = {sum(x for x in data if isinstance(x, int))} from {data}"
```

```
def test_decorator_with_class(prng: MyPRNG):
```

```
    @shuffle_str_ret(prng)
```

```
    class Funcs:
```

```
        def __init__(self, *args):
```

```
            self.args = list(*args)
```

```
        def get_args(self):
```

```
            return self.args
```

```
        def update_args(self, *new_args):
```

```
            self.args.extend(new_args)
```

```
            return self.args
```

```
data = [1, 2, 3, "231", "True"]
```

```
f = Funcs(data)
```

```
assert f.get_args() == data
```

```
new_data = ("Hello", "World")
```

```
res = f.update_args(*new_data)
```

```
data.extend(new_data)
```

```
assert res != data
```

```
# Количество тестов выбирайте самостоятельно,  
# но так, чтобы вся необходимая функциональность  
# была проверена.
```